

Proxy vs Reverse Proxy vs Load Balancer

Interview notes + ready-to-speak answers

1. Forward Proxy

Sits on the **client side** — between a private network (e.g. a company's internal network) and the public internet. It's the middleman for **outgoing** requests; the destination server never talks to the real client directly, only to the proxy.

Analogy: You want a table at a restaurant but don't want to deal with the staff yourself, so you send a personal assistant to make the reservation. The restaurant only ever interacts with the assistant, never with you directly.

What it actually does for a company:

- Filters/blocks malicious sites or scripts before they ever reach an employee's machine
- Lets admins blacklist specific websites for the whole org
- Logs user activity (which sites people are hitting)
- Caches frequently requested content — if one person fetches a resource, the next 10 people get the cached copy instead of hitting the origin again, saving bandwidth

2. Reverse Proxy

Sits on the **server side** — in front of the backend servers. It's the middleman for **incoming** requests, deciding which backend actually handles each one.

Analogy: You arrive at the restaurant and check in at the reception desk. The receptionist (not you) figures out which table is free and walks you to it. The receptionist is a proxy too — just sitting on the receiving end instead of the requesting end.

Core functions (load balancing is only ONE of these):

- **Load balancing** — distributing incoming requests across multiple backend servers
- **Security shield** — instead of exposing every backend server to the internet, you expose only the reverse proxy and harden that one entry point
- **TLS/SSL termination** — decrypts incoming traffic so it can inspect it / make routing decisions
- Caching responses to speed up repeat requests
- Logging for troubleshooting

Most common real-world example: **Nginx**.

3. Why use a Cloud Load Balancer AND a Reverse Proxy together

They are not replacements for each other — in practice you layer both:

- **Cloud load balancer** (e.g. AWS ALB) sits **outside** the private network — it's the first entry point from the public internet, and typically does simple distribution (e.g. least-busy target)
- **Reverse proxy** (e.g. Nginx) sits **inside** the private network and does more fine-grained, intelligent routing

What “more intelligent routing” actually means:

- Sticky sessions — route all requests from the same user to the same backend, based on session/cookie data
- Path-based routing — forward `/orders/*` to one microservice and `/users/*` to another
- Header-based routing decisions, since the reverse proxy can terminate TLS and actually inspect the decrypted request

Layering both also improves security (defense in depth) and scalability — each layer only has to trust the layer immediately in front of it.

Kubernetes parallel:

An **Ingress Controller** in Kubernetes is essentially a reverse proxy living inside the cluster — it handles internal routing/security across services, while the cloud load balancer stays outside as the first line of defense for external traffic.

4. Reverse proxy vs your app's built-in server

Frameworks like Express.js (Node) or Spring Boot (Java) ship with their own lightweight HTTP handling — Spring Boot's embedded Tomcat, for example. That's not the same thing as a dedicated reverse proxy like Nginx.

Nginx is a high-performance web server + reverse proxy: great at serving static content, handling SSL termination, and holding a large number of concurrent connections efficiently. **Express/Spring's embedded server** is meant for the dynamic application logic itself, and is comparatively less efficient at raw connection handling at scale.

Common production pattern: **Nginx in front, app server behind it** — Nginx serves static assets, terminates TLS, and load-balances, while the embedded Tomcat / Express process handles the dynamic request logic. (This is exactly the same role an AWS ALB plays in front of your Spring Boot app's embedded Tomcat — just at the cloud-infra layer instead of the OS-process layer.)

Quick Comparison Table

Aspect	Forward Proxy	Reverse Proxy	Load Balancer
Sits on	Client side (private → internet)	Server side (internet → private)	Server side (internet → private)
Protects	The internal client network	The backend servers	Backend server availability
Client awareness	Destination doesn't see the real client	Client doesn't know which backend served it	Client doesn't know which backend served it
Primary job	Filter outgoing traffic, cache, log	Shield + route + TLS + cache + LB	Distribute traffic across servers
Relationship	Independent concept	Load balancing is ONE feature of this	A feature that lives inside a reverse proxy (or as a standalone cloud service)
Example tool	Squid, company web filters	Nginx, HAProxy, K8s Ingress	AWS ALB/NLB, Nginx, HAProxy

Interview-Ready Spoken Answers

Each answer below is sized for a 30–45 second spoken response — say it out loud, don't read it verbatim.

Q1. What is a forward proxy?

A forward proxy sits between a private network and the public internet, on the client side. It acts as a middleman for outgoing requests — the destination server only ever sees the proxy, not the real client. Companies use it to filter or block malicious traffic, blacklist sites, log activity, and cache frequently requested content so repeat requests don't have to go all the way out to the internet again.

Q2. What is a reverse proxy?

A reverse proxy is the mirror image — it sits in front of backend servers, on the server side, and handles incoming requests. Instead of exposing every backend server directly to the internet, you expose just the reverse proxy and harden that single entry point. Beyond load balancing, it also does TLS termination, caching, logging, and acts as a security shield for everything behind it. Nginx is the classic example.

Q3. What's the actual difference between a proxy and a reverse proxy?

It comes down to which side they sit on and whose identity they're hiding. A forward proxy hides the client from the server — it's used by clients to reach the internet anonymously or under a filtered policy. A reverse proxy hides the server from the client — the client thinks it's talking to one server, but the reverse proxy is actually routing the request to one of many backend servers behind it.

Q4. Is a reverse proxy the same as a load balancer?

Not exactly — load balancing is one feature a reverse proxy can perform, not the whole definition. A reverse proxy can also do TLS termination, caching, security filtering, and logging. So every load balancer is doing

reverse-proxy-style work, but a reverse proxy does a lot more than just balancing load.

Q5. Why would you use both a cloud load balancer and a reverse proxy like Nginx?

Because they solve different problems at different layers. The cloud load balancer, like an AWS ALB, sits outside your private network as the first entry point and does simple, fast distribution. Nginx sits inside the network and does smarter routing — sticky sessions based on cookies, path-based routing to specific microservices, and TLS inspection. Layering both gives you defense in depth and lets each layer specialize instead of asking one component to do everything.

Q6. Where does this fit with a Spring Boot deployment?

Spring Boot ships with an embedded Tomcat, which handles requests fine but isn't built to efficiently hold massive numbers of concurrent connections or serve static assets the way Nginx is. In production, you'd typically put a reverse proxy — or, in a cloud setup, an ALB — in front of that embedded Tomcat to terminate TLS, load balance across instances, and offload anything that doesn't need to touch your actual application logic.